

COMPILADOR DIDÁTICO EM C

Análise Léxica · Sintática · Semântica · Geração de Código

Disciplina: Estrutura de Dados II · ISPTEC

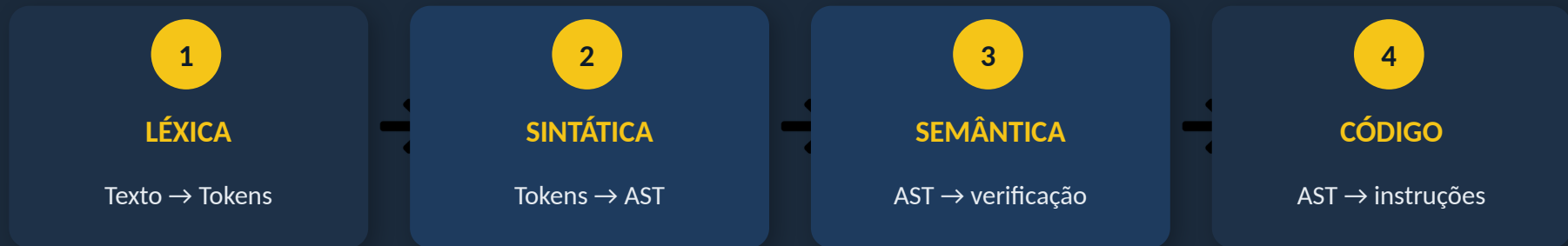
Turma: EINF4 T2 · Grupo N° 2

Njila Camissombo · Ventura Manuel · António Monteiro

2024 / 2025

O Que É Um Compilador?

Um compilador é um programa que traduz código escrito numa linguagem de alto nível para uma representação executável. Cada fase produz uma representação mais abstracta do programa, detectando erros e construindo estruturas de dados essenciais.



Módulos implementados pelo Grupo 2:

Léxico	Parser + Pilha + AST	Semântica + Tabelas + Gerador	Menu + Integração
Elemento 1	Njila Camissombo	Ventura Manuel	António Monteiro

Arquitetura do Sistema

Código fonte (.txt)

Entrada do utilizador

LEXER — tokens[]

Elemento 1

PARSER + PILHA + AST

Njila · verifica estrutura

SEMÂNTICA + TABELAS

Ventura · verifica significado

GERADOR DE CÓDIGO

Ventura · instruções intermédias

MAIN + RELATÓRIO FINAL

António · coordena o fluxo

tipos.h

Estruturas partilhadas

Token tipo, valor, linha

ASTNode tipo, valor, filhos[]

Simbolo nome, tipo, valor

Ficheiros por módulo

- parser.c / parser.h
- stack.c / stack.h
- ast.c / ast.h
- semantic.c / hashtable.c
- bst.c / avl.c
- generator.c / main.c

Gramática Formal da Linguagem

```
programa → main() bloco
bloco → { lista_instrucoes }
instrucao → declaracao_var | atribuicao | print_stmt
          | if_stmt | while_stmt
decl_var → int IDENTIFICADOR ;
atrib → IDENTIFICADOR = expressao ;
print → print ( expressao ) ;
if_stmt → if ( condicao ) bloco
while → while ( condicao ) bloco
condicao → expressao op_rel expressao
op_rel → == | != | < | >
expressao → termo ( + | - ) termo *
termo → IDENTIFICADOR | NUMERO
```

Propriedades

- ✓ **LL(1)**
1 token de lookahead
- ✓ **Sem recursão esq.**
necessário para parser recursivo
- ✓ **Sem ambiguidade**
1 derivação por sequência

Exemplos

- ✓ `int x;` declaração válida
- ! `int ;` falta identificador
- ✓ `x = a + 5;` atribuição válida
- ! `print(x}` par errado na pilha

Módulo Parser + Pilha — Análise Sintática

Parser Descendente Recursivo

- Uma função C por regra da gramática
- `peek()` — espia o token actual sem avançar
- `consume(tipo)` — verifica tipo e avança
- Em erro: mostra linha, esperado e encontrado

```
parse_atribuicao() → 'x = a + 5;'  
  consume(IDENT) → consome 'x'  
  consume(IGUAL) → consome '='  
  parse_expressao()  
    parse_termo() → consome 'a'  
    consume(MAIS) → consome '+'  
    parse_termo() → consome '5'  
    consume(PT_VIRG) → consome ';' 
```

Pilha de Delimitadores

Garante que `() {}` abrem e fecham correctamente

`print(x);`



OK

`print(x}`



ERRO

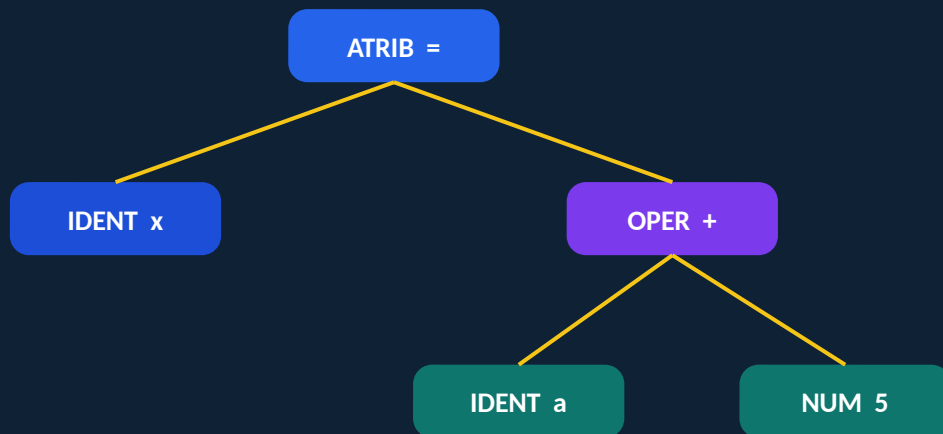
Funções da pilha

- `pilha_init()`
- `pilha_push(p, c)`
- `pilha_pop(p)`
- `pilha_vazia(p)`
- `pilha_verificar_fim(p)`

AST — Árvore Sintática Abstrata

Representa a estrutura hierárquica do programa após a análise sintática. É o resultado entregue ao módulo semântico. Cada nó tem: tipo, valor e até 10 filhos.

`x = a + 5;`



Funções implementadas

`ast_criar_no(tipo, valor)`

Cria novo nó

`ast_adicionar_filho(pai, f)`

Liga filho ao pai

`ast_imprimir(no, prof)`

Imprime a árvore

`ast_libertar(no)`

Liberta memória

Tipos de nós

- NO_PROGRAMA
- NO_DECLARACAO_VAR
- NO_ATRIBUICAO
- NO_OPERACAO
- NO_IDENTIFICADOR
- NO_NUMERO
- NO_PRINT
- NO_IF
- NO_WHILE

Análise Semântica e Tabela de Símbolos

O que verifica

⚠ **Variável não declarada**
`x = 10; (sem int x)`

⚠ **Declaração duplicada**
`int x; int x;`

⚠ **Tipos incompatíveis**
`int x; x = "texto";`

```
/* Código com erro */  
main() {  
    x = 10;    /* sem declaração */  
}
```

```
/* Mensagem gerada */  
[SEMÂNTICA] ERRO na linha 3:  
variável 'x' não foi declarada
```

Hash Table

$O(1)$ médio

Inserção/pesquisa rápida

BST

$O(\log n)$ médio

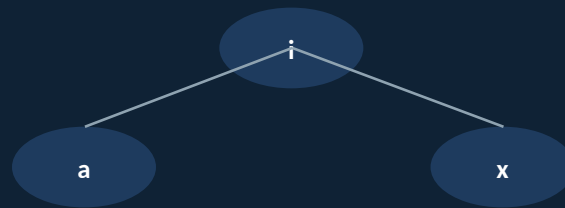
Símbolos em ordem alfab.

AVL

$O(\log n)$ garantido

BST auto-balanceada

BST da tabela de símbolos



Pesquisa $O(\log n)$

Gerador de Código Intermédio

Recebe a AST anotada e produz instruções de código intermédio de três endereços — independentes da arquitectura do processador. Base para optimizações e tradução para código máquina.

Conjunto de instruções

LOAD fonte Carrega valor no acumulador

STORE destino Guarda acumulador na variável

ADD / SUB oper. Soma ou subtrai

PRINT fonte Imprime o valor

CMP op1 op2 Compara (para if/while)

JMP/JEQ/JLT label Saltos condicionais

LABEL nome Define ponto de salto

```
/* x = a + 5; */  
LOAD a  
ADD 5  
STORE x
```

```
/* print(x); */  
PRINT x
```

```
/* while (i < 10) { i = i + 1; } */  
LABEL while_inicio  
CMP i 10  
JLT while_corpo  
JMP while_fim  
LABEL while_corpo  
LOAD i
```


Programa Exemplo — Fluxo Completo

```
main() {  
    int x; int a; int i;  
    a = 3;  
    x = a + 5;  
    i = 0;  
    while (i < 10) { i = i + 1; }  
    if (x == 8) { print(x); }  
}
```

RELATÓRIO DE COMPILAÇÃO

Tokens processados:	47
Erros léxicos:	0
Erros sintáticos:	0
Erros semânticos:	0

Símbolos declarados:
x → int

1

Lexer

47 tokens extraídos

2

Parser

AST com 14 nós construída

3

Semântica

3 símbolos registados

4

Gerador

Código intermédio gerado

Testes — Detecção de Erros

SINTÁTICO



Entrada:

```
int ;
```

Mensagem de erro:

```
[PARSER] ERRO linha 1: esperado  
IDENT, encontrado ';' ;
```

PILHA



Entrada:

```
print(x}
```

Mensagem de erro:

```
[PILHA] ERRO: '}' sem '{'  
correspondente
```

SEMÂNTICO



Entrada:

```
x = 10; (sem int x)
```

Mensagem de erro:

```
[SEM] ERRO linha 1: variável 'x'  
não declarada
```

SINTÁTICO



Entrada:

```
x = a + ;
```

Mensagem de erro:

```
[PARSER] ERRO linha 1: esperado IDENT  
ou NUMERO
```

PILHA



Entrada:

```
{ int x;
```

Mensagem de erro:

```
[PILHA] ERRO: '{' por fechar no fim  
do programa
```

SEMÂNTICO



Entrada:

```
int x; int x;
```

Mensagem de erro:

```
[SEM] ERRO linha 2: variável 'x'  
já declarada
```

Conclusões

- ✓ Compilador modular com 4 fases: léxica, sintática, semântica e geração de código
- ✓ Parser descendente recursivo — clareza e facilidade de depuração
- ✓ Pilha garante 100% de detecção de erros de delimitadores
- ✓ AST como interface limpa entre front-end e back-end
- ✓ Tabela de símbolos com 3 estruturas (Hash, BST, AVL) para diferentes necessidades
- ✓ Todos os casos de teste detectados correctamente — 0 falsos negativos

Referências bibliográficas

- CORMEN et al. Introduction to Algorithms. 4ª ed. MIT Press, 2022. ISBN 978-0-262-04630-5.
- AHO, HOPCROFT, ULLMAN. Data Structures and Algorithms. Addison-Wesley, 1983.
- BULLINARIA. Lecture Notes for Data Structures and Algorithms. Univ. of Birmingham, 2019.

Obrigado!

Disponíveis para perguntas

EINF4 T2 · Grupo N° 2

Njila Camissombo · Ventura Manuel · António Monteiro